**ST10492775**

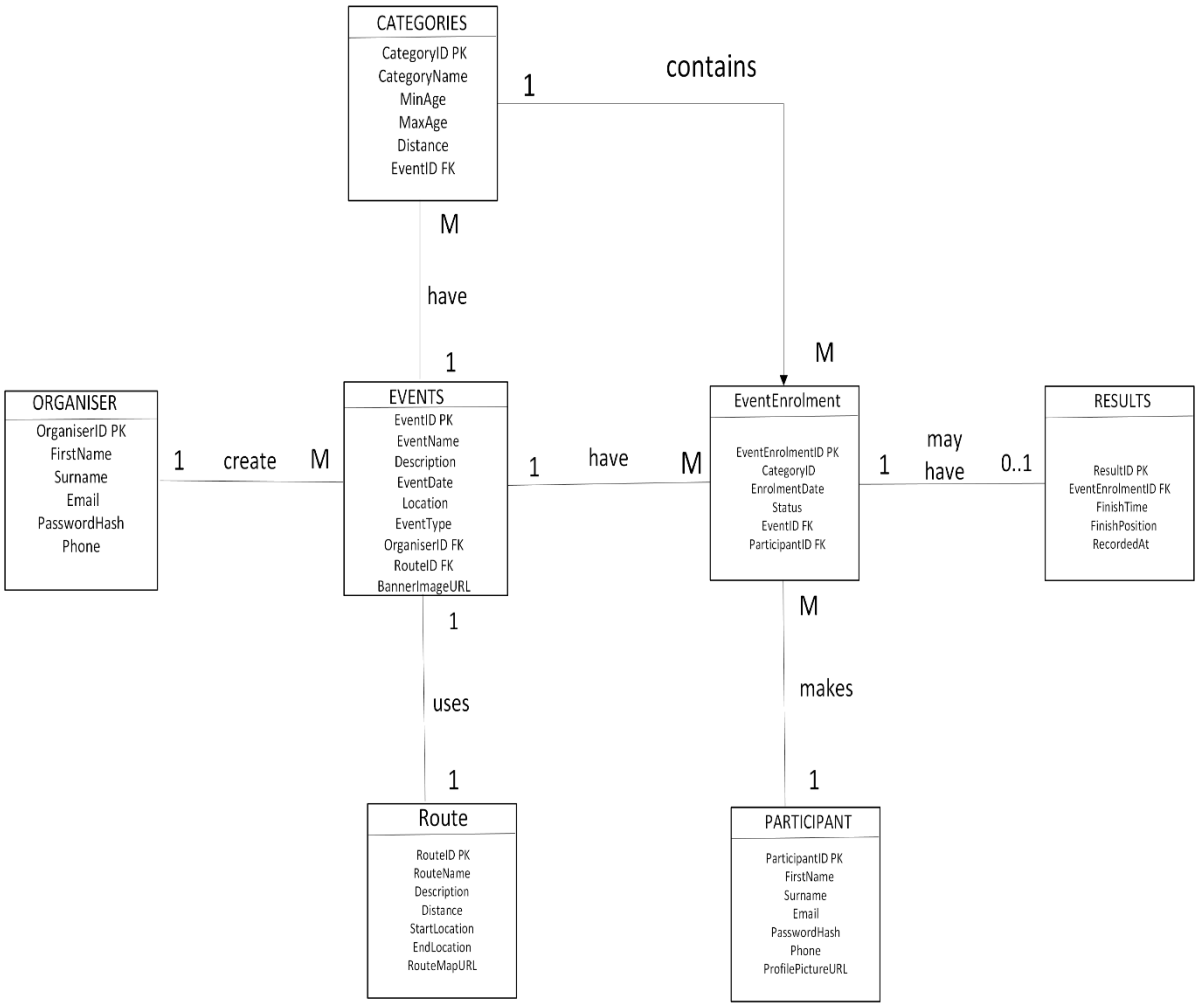
**TUWADUVHANI LONDOTANI**

**PROG6212**

**PART 1**

# PART 1

# ENTITY RELATIONSHIO DIAGRAM




**ERD Summary**

The Race Day ERD contains **7 main entities**:

- **Organiser** – creates and manages events. **PK: OrganiserID**
- **Participant** – stores participant information. **PK: ParticipantID**
- **Events** – stores race event information. **PK: EventID; FK: OrganiserID, RouteID**
- **Category** – defines age groups and distances for an event. **PK: CategoryID; FK: EventID**
- **EventEnrolment** – records participants registering for events and categories. **PK: EventEnrolmentID; FK: EventID, ParticipantID, CategoryID**

- **Results** – stores participants' race results. **PK: ResultID; FK: EventEnrolmentID**
- **Route** – stores the route used by an event. **PK: RouteID**

**Main Relationships**

- **Organiser 1:M Events** – one organiser can create many events.
- **Events 1:M Category** – one event can have many categories.
- **Events 1:M EventEnrolment** – one event can have many enrolments.
- **Category 1:M EventEnrolment** – one category can have many enrolments.
- **Participant 1:M EventEnrolment** – one participant can have many enrolments.
- **EventEnrolment 1:0..1 Results** – an enrolment may have no result or one result.
- **Events 1:1 Route** – each event uses one route.

**Overall Functionality**

The system allows organisers to create and manage events, assign categories and routes to events, and allows participants to enrol in a specific event and category. The EventEnrolment entity connects the participant, event and category. Once the participant completes the race, their finish time and position can be recorded in Results.

# API ENDPOINT PLAN

| HTTP METHOD | ROUTE | DESCRIPTION | ROLE REQUIRED | REQUEST BODY | EXPECTED RESPONSE |
|---|---|---|---|---|---|
| **POST** | /api/auth/register | Registers a new participant or organiser account. | None | { firstName, surname, email, password, role } | **201 Created** – account created. **400 Bad Request** – invalid details. |
| **POST** | /api/auth/login | Authenticates a user and provides an access token. | None | { email, password } | **200 OK** – token and user details. **401 Unauthorized** – invalid credentials. |
| **GET** | /api/profile | Retrieves the profile of the currently logged-in user. | Participant / Organiser | None | **200 OK** – profile details. **401 Unauthorized** – not logged in. |
| **PUT** | /api/profile | Updates the currently logged-in user's profile. | Participant / Organiser | { firstName, surname, phone, … } | **200 OK** – updated profile. **400 Bad Request** – invalid data. |
| **GET** | /api/events | Retrieves all available race events. | Participant / Organiser | None | **200 OK** – list of events. |
| **GET** | /api/events/{id} | Retrieves details of a specific event. | Participant / Organiser | None | **200 OK** – event details. **404 Not Found** – event does not exist. |
| **POST** | /Api/events | Creates a new race event. | Organiser | { eventName, description, eventDate, location, eventType, routeId } | **201 Created** – event created. **400 Bad Request** – invalid data. |
| **PUT** | /api/events/{id} | Updates an existing race event. | Organiser | { eventName, description, eventDate, location, eventType, routeId } | **200 OK** – event updated. **404 Not Found** – event does not exist. |

| | | | | | |
|---|---|---|---|---|---|
| **DELETE** | /api/events/{id} | Deletes an event. | Organiser | None | **204 No Content** – event deleted. **404 Not Found** – event does not exist. |
| **GET** | api/categories | Retrieves available race categories. | Participant / Organiser | None | **200 OK** – list of categories. |
| **GET** | /api/categories/{id} | Retrieves a specific race category. | Participant / Organiser | None | **200 OK** – category details. **404 Not Found** – category not found. |
| **POST** | /api/categories | Creates a new event category. | Organiser | { eventId, categoryName, minAge, maxAge, distance } | **201 Created** – category created. **400 Bad Request** – invalid data. |
| **PUT** | /api/categories/{id} | Updates an existing category. | Organiser | { categoryName, minAge, maxAge, distance } | **200 OK** – category updated. **404 Not Found** – category not found. |
| **DELETE** | /api/categories/{id} | Deletes a category. | Organiser | None | **204 No Content** – category deleted. **404 Not Found** – category not found. |
| **GET** | /api/routes/{id} | Retrieves the route information for a race. | Participant / Organiser | None | **200 OK** – route details. **404 Not Found** – route not found. |
| **POST** | /api/routes | Creates a new race route. | Organiser | { routeName, description, distance, startLocation, endLocation, routeMapUrl } | **201 Created** – route created. |
| **PUT** | /api/routes/{id} | Updates an existing race route. | Organiser | { routeName, description, distance, startLocation, endLocation, routeMapUrl } | **200 OK** – route updated. **404 Not Found** – route not found. |

| | | | | | |
|---|---|---|---|---|---|
| **DELETE** | /api/routes/{id} | Deletes a race route. | Organiser | None | **204 No Content** – route deleted. **404 Not Found** – route not found. |
| **POST** | /api/enrolments | Allows a participant to enrol in an event and category. | Participant | { eventId, categoryId } | **201 Created** – enrolment created. **400 Bad Request** – invalid enrolment. **409 Conflict** – already enrolled. |
| **GET** | /api/enrolments | Retrieves enrolments belonging to the logged-in participant. | Participant | None | **200 OK** – list of participant's enrolments. |
| **GET** | /api/events/{eventId}/ Enrolments | Retrieves participants enrolled in a specific event. | Organiser | None | **200 OK** – list of enrolments. **404 Not Found** – event not found. |
| **GET** | /api/enrolments/{id} | Retrieves details of a specific enrolment | Participant / Organiser | None | **200 OK** – enrolment details. **404 Not Found** – enrolment not found. |
| **DELETE** | /api/enrolments/{id} | Cancels a participant's enrolment. | Participant | None | **204 No Content** – enrolment cancelled. **404 Not Found** – enrolment not found. |
| **GET** | /api/results | Retrieves race results. | Participant / Organiser | None | **200 OK** – list of results. |
| **GET** | /api/results/{id} | Retrieves a specific race result. | Participant / Organiser | None | **200 OK** – result details. **404 Not Found** – result not found. |

| | | | | | |
|---|---|---|---|---|---|
| **POST** | /api/results<br>Organiser | Records a participant's race result. | Organiser | { eventEnrolmentId, finishTime, finishPosition } | **201 Created** – result recorded. **400 Bad Request** – invalid data. |
| **PUT** | /api/results/{id} | Updates an existing race result. | Organiser | { finishTime, finishPosition } | **200 OK** – result updated. **404 Not Found** – result not found. |

This is API Endpoint Plan for the RaceDay system. It shows how the frontend will communicate with the backend.

The plan uses four main HTTP methods: **GET** to retrieve data, **POST** to create data, **PUT** to update data, and **DELETE** to remove data.

The API covers **authentication, user profiles, events, categories, event enrolments, results and routes**.

For authentication, users can **register and log in**. Both organisers and participants can manage their profiles.

Organisers can **create, update and delete events, categories and routes**, while participants can view events and categories.

Participants can use the enrolment endpoints to **register for events and view or cancel their enrolments**. Organisers can view the participants enrolled in their events.

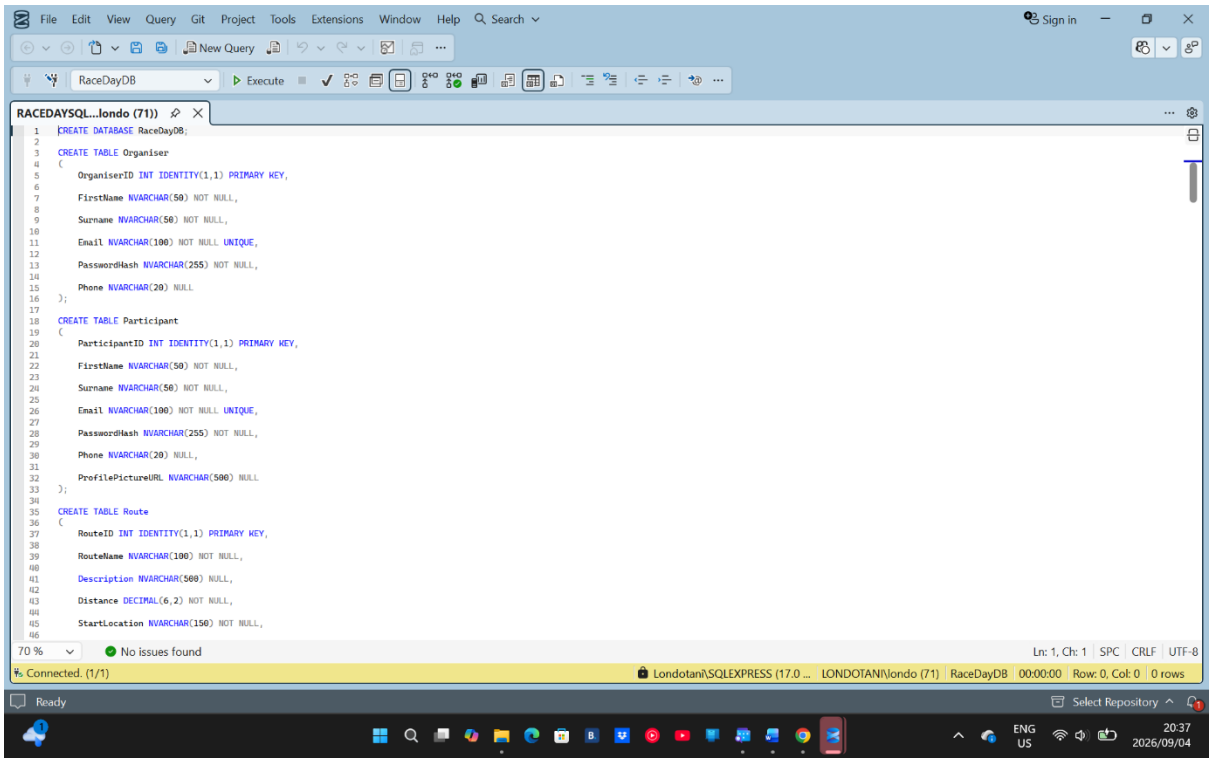
The results endpoints allow users to **view results**, while organisers can **record and update results**.

The API also matches my ERD. For example, **EventEnrolment connects Participant, Event and Category**, while **Results is connected to EventEnrolment**.

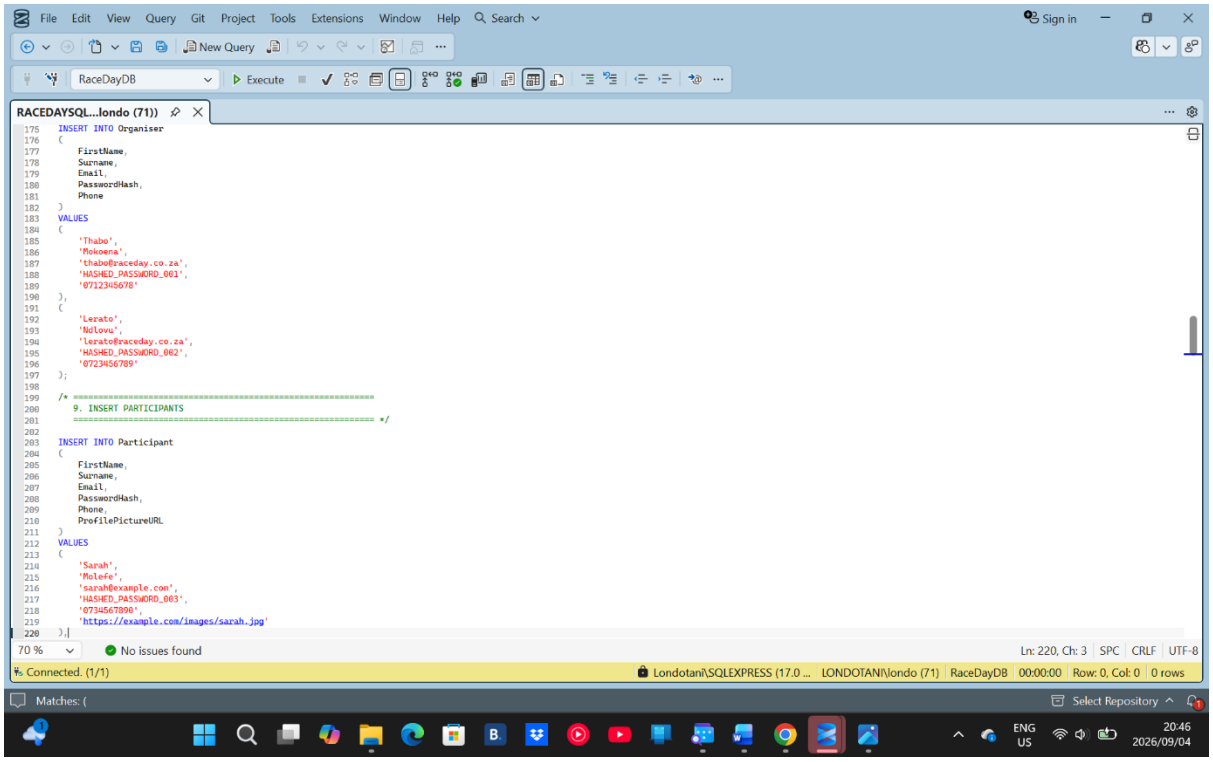
The **Role Required** column controls what each type of user is allowed to do.

Overall, the API Endpoint Plan provides the structure for implementing the RaceDay system and ensures that the API matches the database design

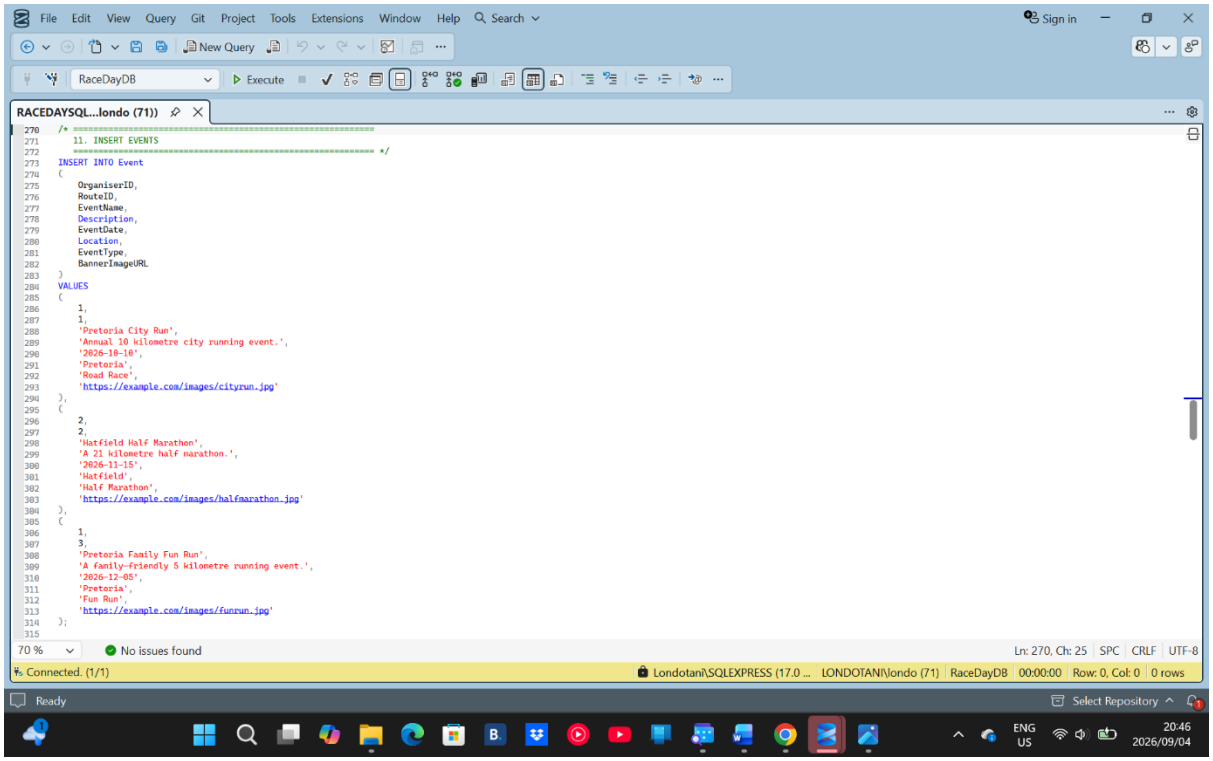
# SQL



```sql
CREATE DATABASE RaceDayDB;

CREATE TABLE Organiser
(
    OrganiserID INT IDENTITY(1,1) PRIMARY KEY,

    FirstName NVARCHAR(50) NOT NULL,

    Surname NVARCHAR(50) NOT NULL,

    Email NVARCHAR(100) NOT NULL UNIQUE,

    PasswordHash NVARCHAR(255) NOT NULL,

    Phone NVARCHAR(20) NULL
);

CREATE TABLE Participant
(
    ParticipantID INT IDENTITY(1,1) PRIMARY KEY,

    FirstName NVARCHAR(50) NOT NULL,

    Surname NVARCHAR(50) NOT NULL,

    Email NVARCHAR(100) NOT NULL UNIQUE,

    PasswordHash NVARCHAR(255) NOT NULL,

    Phone NVARCHAR(20) NULL,

    ProfilePictureURL NVARCHAR(500) NULL
);

CREATE TABLE Route
(
    RouteID INT IDENTITY(1,1) PRIMARY KEY,

    RouteName NVARCHAR(100) NOT NULL,

    Description NVARCHAR(500) NULL,

    Distance DECIMAL(6,2) NOT NULL,

    StartLocation NVARCHAR(150) NOT NULL,
```



```sql
INSERT INTO Organiser
(
    FirstName,
    Surname,
    Email,
    PasswordHash,
    Phone
)
VALUES
(
    'Thabo',
    'Mokoena',
    'thabo@raceday.co.za',
    'HASHED_PASSWORD_001',
    '0712345678'
),
(
    'Lerato',
    'Ndlovu',
    'lerato@raceday.co.za',
    'HASHED_PASSWORD_002',
    '0723456789'
);

/* ============================================================
   9. INSERT PARTICIPANTS
   ============================================================ */

INSERT INTO Participant
(
    FirstName,
    Surname,
    Email,
    PasswordHash,
    Phone,
    ProfilePictureURL
)
VALUES
(
    'Sarah',
    'Molefe',
    'sarah@example.com',
    'HASHED_PASSWORD_003',
    '0734567890',
    'https://example.com/images/sarah.jpg'
),
```




CREATE DATABASE RaceDayDB;

CREATE TABLE Organiser

(

OrganiserID INT IDENTITY(1,1) PRIMARY KEY,

FirstName NVARCHAR(50) NOT NULL,

Surname NVARCHAR(50) NOT NULL,

Email NVARCHAR(100) NOT NULL UNIQUE,

PasswordHash NVARCHAR(255) NOT NULL,

Phone NVARCHAR(20) NULL

```
);

CREATE TABLE Participant
(
   ParticipantID INT IDENTITY(1,1) PRIMARY KEY,

   FirstName NVARCHAR(50) NOT NULL,

   Surname NVARCHAR(50) NOT NULL,

   Email NVARCHAR(100) NOT NULL UNIQUE,

   PasswordHash NVARCHAR(255) NOT NULL,

   Phone NVARCHAR(20) NULL,

   ProfilePictureURL NVARCHAR(500) NULL
);

CREATE TABLE Route
(
   RouteID INT IDENTITY(1,1) PRIMARY KEY,

   RouteName NVARCHAR(100) NOT NULL,

   Description NVARCHAR(500) NULL,

   Distance DECIMAL(6,2) NOT NULL,
```

```
    StartLocation NVARCHAR(150) NOT NULL,

    EndLocation NVARCHAR(150) NOT NULL,

    RouteMapURL NVARCHAR(500) NULL,

    CONSTRAINT CK_Route_Distance
        CHECK (Distance > 0)
);

CREATE TABLE Event
(
    EventID INT IDENTITY(1,1) PRIMARY KEY,

    OrganiserID INT NOT NULL,

    RouteID INT NOT NULL,

    EventName NVARCHAR(150) NOT NULL,

    Description NVARCHAR(500) NULL,

    EventDate DATE NOT NULL,

    Location NVARCHAR(150) NOT NULL,

    EventType NVARCHAR(50) NOT NULL,
```

```
    BannerImageURL NVARCHAR(500) NULL,

    CONSTRAINT FK_Event_Organiser
        FOREIGN KEY (OrganiserID)
        REFERENCES Organiser(OrganiserID),

    CONSTRAINT FK_Event_Route
        FOREIGN KEY (RouteID)
        REFERENCES Route(RouteID)
);

CREATE TABLE Categories
(
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,

    EventID INT NOT NULL,

    CategoryName NVARCHAR(100) NOT NULL,

    MinAge INT NOT NULL,

    MaxAge INT NOT NULL,

    Distance DECIMAL(6,2) NOT NULL,

    CONSTRAINT FK_Categories_Event
        FOREIGN KEY (EventID)
```

```
    REFERENCES Event(EventID),

  CONSTRAINT CK_Categories_Age
    CHECK (MinAge >= 0 AND MaxAge >= MinAge),

  CONSTRAINT CK_Categories_Distance
    CHECK (Distance > 0),

  CONSTRAINT UQ_Categories_Event_Name
    UNIQUE (EventID, CategoryName)
);



CREATE TABLE EventEnrolment
(
  EventEnrolmentID INT IDENTITY(1,1) PRIMARY KEY,

  EventID INT NOT NULL,

  ParticipantID INT NOT NULL,

  CategoryID INT NOT NULL,

  EnrolmentDate DATE NOT NULL
    CONSTRAINT DF_EventEnrolment_Date
    DEFAULT CAST(GETDATE() AS DATE),

  Status NVARCHAR(30) NOT NULL
```

```
    CONSTRAINT DF_EventEnrolment_Status
    DEFAULT 'Confirmed',

  CONSTRAINT FK_EventEnrolment_Event
    FOREIGN KEY (EventID)
    REFERENCES Event(EventID),

  CONSTRAINT FK_EventEnrolment_Participant
    FOREIGN KEY (ParticipantID)
    REFERENCES Participant(ParticipantID),

  CONSTRAINT FK_EventEnrolment_Category
    FOREIGN KEY (CategoryID)
    REFERENCES Categories(CategoryID),

  CONSTRAINT CK_EventEnrolment_Status
    CHECK (Status IN ('Confirmed', 'Cancelled', 'Pending')),

  CONSTRAINT UQ_EventEnrolment_Participant_Event
    UNIQUE (ParticipantID, EventID)
);

CREATE TABLE Results
(
  ResultID INT IDENTITY(1,1) PRIMARY KEY,

  EventEnrolmentID INT NOT NULL,
```

```
    FinishTime TIME(0) NOT NULL,

    FinishPosition INT NOT NULL,

    RecordedAt DATETIME2 NOT NULL
        CONSTRAINT DF_Results_RecordedAt
        DEFAULT GETDATE(),

    CONSTRAINT FK_Results_EventEnrolment
        FOREIGN KEY (EventEnrolmentID)
        REFERENCES EventEnrolment(EventEnrolmentID),

    CONSTRAINT UQ_Results_EventEnrolment
        UNIQUE (EventEnrolmentID),

    CONSTRAINT CK_Results_Position
        CHECK (FinishPosition > 0)
);

INSERT INTO Organiser
(
    FirstName,
    Surname,
    Email,
    PasswordHash,
    Phone
)
VALUES
```

```
(
    'Thabo',
    'Mokoena',
    'thabo@raceday.co.za',
    'HASHED_PASSWORD_001',
    '0712345678'
),
(
    'Lerato',
    'Ndlovu',
    'lerato@raceday.co.za',
    'HASHED_PASSWORD_002',
    '0723456789'
);

/* ============================================================
   9. INSERT PARTICIPANTS
   ============================================================ */

INSERT INTO Participant
(
    FirstName,
    Surname,
    Email,
    PasswordHash,
    Phone,
    ProfilePictureURL
)
```

```
VALUES
(
    'Sarah',
    'Molefe',
    'sarah@example.com',
    'HASHED_PASSWORD_003',
    '0734567890',
    'https://example.com/images/sarah.jpg'
),
(
    'Daniel',
    'Mthembu',
    'daniel@example.com',
    'HASHED_PASSWORD_004',
    '0745678901',
    'https://example.com/images/daniel.jpg'
);

/* ============================================================
   10. INSERT ROUTES
   ============================================================ */

INSERT INTO Route
(
    RouteName,
    Description,
    Distance,
    StartLocation,
```

```
    EndLocation,
    RouteMapURL
)
VALUES
(
    'Pretoria City Route',
    'Road running route through Pretoria city.',
    10.00,
    'Church Square',
    'Union Buildings',
    'https://example.com/routes/pretoria10km'
),
(
    'Hatfield Challenge Route',
    'Running route around Hatfield.',
    21.10,
    'Hatfield',
    'University of Pretoria',
    'https://example.com/routes/hatfield21km'
),
(
    'Pretoria Fun Run Route',
    'Short family-friendly running route.',
    5.00,
    'Pretoria CBD',
    'Church Square',
    'https://example.com/routes/funrun5km'
);
```

```
/* ============================================================
   11. INSERT EVENTS
   ============================================================ */
INSERT INTO Event
(
    OrganiserID,
    RouteID,
    EventName,
    Description,
    EventDate,
    Location,
    EventType,
    BannerImageURL
)
VALUES
(
    1,
    1,
    'Pretoria City Run',
    'Annual 10 kilometre city running event.',
    '2026-10-10',
    'Pretoria',
    'Road Race',
    'https://example.com/images/cityrun.jpg'
),
(
```

```
    2,
    2,
    'Hatfield Half Marathon',
    'A 21 kilometre half marathon.',
    '2026-11-15',
    'Hatfield',
    'Half Marathon',
    'https://example.com/images/halfmarathon.jpg'
),
(
    1,
    3,
    'Pretoria Family Fun Run',
    'A family-friendly 5 kilometre running event.',
    '2026-12-05',
    'Pretoria',
    'Fun Run',
    'https://example.com/images/funrun.jpg'
);




/* ============================================================
   12. INSERT CATEGORIES
   Categories for EACH of the 3 events
   ============================================================ */
```

```
INSERT INTO Categories
(
   EventID,
   CategoryName,
   MinAge,
   MaxAge,
   Distance
)
VALUES

-- EVENT 1
(
   1,
   'Junior',
   13,
   17,
   10.00
),
(
   1,
   'Senior',
   18,
   39,
   10.00
),
(
   1,
   'Veteran',
```

```
    40,
    100,
    10.00
),

-- EVENT 2
(
    2,
    'Open',
    18,
    39,
    21.10
),
(
    2,
    'Veteran',
    40,
    100,
    21.10
),

-- EVENT 3
(
    3,
    'Junior',
    10,
    17,
    5.00
```

```
),
(
   3,
   'Adult',
   18,
   59,
   5.00
),
(
   3,
   'Senior',
   60,
   100,
   5.00
);

 --============================================================
 --  13. INSERT EVENT ENROLMENTS
 -- ============================================================

INSERT INTO EventEnrolment
(
   EventID,
   ParticipantID,
   CategoryID,
   EnrolmentDate,
   Status
)
```

```
VALUES
(
    1,
    1,
    2,
    '2026-09-01',
    'Confirmed'
),
(
    1,
    2,
    2,
    '2026-09-02',
    'Confirmed'
),
(
    2,
    1,
    4,
    '2026-09-03',
    'Confirmed'
),
(
    3,
    2,
    7,
    '2026-09-03',
    'Confirmed'
```

```
);

/* ============================================================
   14. INSERT SAMPLE RESULTS
   ============================================================ */

INSERT INTO Results
(
    EventEnrolmentID,
    FinishTime,
    FinishPosition
)
VALUES
(
    1,
    '00:52:35',
    1
),
(
    2,
    '00:58:42',
    2
);

/* ============================================================
   15. TEST THE DATABASE
   ============================================================ */
```

```
SELECT * FROM Organiser;

SELECT * FROM Participant;

SELECT * FROM Route;

SELECT * FROM Event;

SELECT * FROM Categories;

SELECT * FROM EventEnrolment;

SELECT * FROM Results;
```